ONEDRAFT

CAD-AI — Aria Powered

Foundational Technical Specification

Version 1.0

Document Status: Foundational Architecture
System: OneDraft / CAD-AI
Engine: Aria Powered
Primary Output: Coordinated Architectural Documentation
Drawing Standard: ARCH D / jurisdiction-configurable
Model Principle: Single Source of Truth

---

1. PURPOSE

OneDraft shall operate as a model-driven architectural drafting platform capable of transforming a customer's detailed project specification into a coordinated architectural project model and associated drawing documentation.

The Project Data Model is the authoritative internal representation of every OneDraft project.

Drawings shall not constitute the primary source of project information. Rather, drawings are generated representations of the underlying project model.

Accordingly, the fundamental architecture is:

PROJECT DATA → PROJECT MODEL → GEOMETRY → DOCUMENTATION

and not:

PROMPT → IMAGE

The system must therefore understand a building as a coordinated collection of spaces, elements, assemblies, systems, relationships, constraints, requirements and documentation.

---

2. PROJECT IDENTITY

Every OneDraft project receives a unique immutable `Project_ID`.

The project identity record shall contain:

Project_ID
Project_Name
Project_Type
Project_Status
Client_ID
Organization_ID
Project_Address
Jurisdiction
Municipality
Province/State
Country
Geographic_Coordinates
Units
Currency
Created_Date
Last_Modified_Date
Current_Revision
Current_Model_Version
Current_Code_Manifest
Current_Document_Set

The Project_ID shall remain constant throughout the life of the project.

Revisions shall create new versions of the project state rather than overwrite historical project states.

---

3. PROJECT CLASSIFICATION

The system shall classify the project before architectural generation.

The classification structure shall support, at minimum:

Residential
Multi-Residential
Commercial
Office
Retail
Industrial
Institutional
Educational
Healthcare
Hospitality
Assembly
Sports / Stadium

Mixed Use
Infrastructure
Renovation
Addition
Tenant Improvement
Accessibility Modification
Custom / Other

Project classification shall influence subsequent code analysis, drawing requirements, spatial rules and documentation requirements.

---

4. PROJECT BRIEF

The Project Brief represents the customer's human-readable requirements.

It shall preserve the original customer input rather than replacing it with AI interpretation.

The Project Brief shall contain:

Original Specification

Customer Requirements

Functional Requirements

Spatial Requirements

Aesthetic Requirements

Material Preferences

Performance Requirements

Accessibility Requirements

Environmental Requirements

Budget Parameters

Schedule Parameters

Site Constraints

Known Regulatory Constraints

Customer Exclusions

Customer Priorities

Special Instructions

Every original customer instruction shall be retained in the project record.

The AI interpretation of that instruction shall be stored separately.

This creates an important distinction:

Customer Input ≠ AI Interpretation

The system must preserve both.

---

5. REQUIREMENTS MODEL

The Project Brief shall be transformed into a structured Requirements Model.

Each requirement receives a unique:

`Requirement_ID`

Each requirement shall contain:

Requirement_ID

Source

Description

Requirement_Type

Priority

Mandatory / Preferred

Applicable_Scope

Related_Elements

Verification_Status

Conflict_Status

Resolution_Status

Revision_History

For example:

`REQ-000147`

> Main-floor accessible bedroom required.

The system can then associate that requirement with:

`Room → Bedroom 01`

and subsequently verify that the resulting room satisfies the applicable dimensional and accessibility constraints.

6. SITE MODEL

Every project shall contain a Site Model.

The Site Model represents the physical context within which the project exists.

It shall contain:

Property Boundary

Lot Dimensions

Legal Description

Site Coordinates

North Orientation

Elevation Data

Existing Structures

Existing Utilities

Road Access

Pedestrian Access

Parking

Landscaping

Topography

Drainage Information

Adjacent Properties

Setbacks

Easements

Rights-of-Way

Environmental Constraints

Survey References

Site Photos / References

Planning Constraints

The Site Model shall be geometrically independent from the building model but spatially linked to it.

7. BUILDING MODEL

The Building Model is the central architectural representation.

A building shall be represented as a hierarchical structure:

Project

→ **Site**

→ **Building**

→ **Building Zone**

→ **Level**

→ **Space**

→ **Element**

→ **Assembly**

→ **Component**

This hierarchy allows the system to understand the building at different levels of abstraction.

---

8. LEVEL MODEL

Each physical level shall receive a unique:

`Level_ID`

Examples:

`P2`

`P1`

`G`

`L1`

`L2`

`L3`

`ROOF`

`TRUSS`

A level record shall contain:

Level_ID

Level_Name

Elevation

Floor-to-Floor Height

Structural Datum

Finished Floor Elevation

Ceiling Elevation

Level Function

Applicable Zones

Associated Drawing Sheets

The model must not treat levels merely as drawing layers.

A level is a physical building datum from which geometry and relationships are derived.

---

9. SPATIAL MODEL

Every usable space shall receive a unique:

`Space_ID`

Examples:

`RM-001`

`RM-002`

`COR-001`

`STAIR-001`

`MECH-001`

`ELEC-001`

Each space shall contain:

Space_ID

Room Name

Room Number

Occupancy Type

Level

Area

Volume

Boundary Geometry

Ceiling Height

Required Clearances

Accessibility Classification

Fire/Life-Safety Classification

Adjacent Spaces

Doors

Windows

Mechanical Requirements

Electrical Requirements

Plumbing Requirements

Finish Requirements

Applicable Requirements

This allows the AI to reason about rooms rather than simply draw rectangles labelled "Bedroom."

---

10. ARCHITECTURAL ELEMENT MODEL

Physical architectural elements shall receive persistent identities.

Examples include:

Walls

Doors

Windows

Stairs

Ramps

Guardrails

Handrails

Columns

Slabs

Roofs

Ceilings

Partitions

Curtain Walls

Millwork

Casework

Fixtures

Equipment

Each element shall contain:

Element_ID

Element_Type

Geometry

Level

Host

Material

Assembly

Dimensions

Location

Orientation

Relationships

Requirements

Code Constraints

Drawing References

Revision History

The persistent identity is critical.

If `WALL-00427` appears on a floor plan, section and elevation, all three representations reference the same underlying element.

---

11. GEOMETRIC MODEL

OneDraft shall maintain a geometric representation separate from the visual drawing representation.

Geometry shall support:

Points

Lines

Arcs

Polylines

Curves

Surfaces

Solids

Meshes where required

Planes

Datums

Coordinate Systems

Bounding Volumes

The geometric engine shall support parametric relationships.

For example:

`Wall_A`

is constrained to:

`Grid_03`

and offset from:

`Grid_03 = 1500 mm`

Changing the grid shall therefore permit dependent geometry to update automatically.

---

12. PARAMETRIC CONSTRAINT MODEL

Constraints shall be first-class project objects.

Constraint types shall include:

Dimensional

Geometric

Alignment

Parallel

Perpendicular

Concentric

Equal

Offset

Minimum

Maximum

Clearance

Adjacency

Containment

Accessibility

Code-derived

Structural

Mechanical

Customer-defined

A constraint shall contain:

Constraint_ID

Constraint_Type

Source

Affected_Elements

Value

Unit

Tolerance

Priority

Status

Conflict Information

This allows OneDraft to distinguish between a drawing that merely looks correct and a model that is mathematically constrained.

---

13. ASSEMBLY MODEL

Construction assemblies shall be represented independently from individual geometry.

For example:

Exterior Wall Assembly

may contain:

Finish
Air Barrier
Sheathing
Studs
Insulation
Vapour Control
Interior Gypsum

The assembly record shall contain:

Assembly_ID

Assembly_Type

Layer Sequence

Material Properties

Thickness

Performance Properties

Fire Rating

Thermal Properties

Acoustic Properties

Manufacturer Data where applicable

Applicable Code Requirements

Assemblies may subsequently feed wall sections, details, schedules and specifications automatically.

---

14. CODE CONTROL MODEL

Before production drawing generation, OneDraft shall establish a Code Control Model.

The Code Control Model shall identify the regulatory framework applicable to the project.

It shall contain:

Jurisdiction

Municipality

Province/State

Country

Building Code

Fire Code

Accessibility Requirements

Energy Requirements

Electrical Requirements

Plumbing Requirements

Mechanical Requirements

Zoning Regulations

Planning Requirements

Municipal Amendments

Applicable Standards

Effective Dates

Source References

Version Identifiers

Retrieval Dates

The system shall never represent the code state as an unqualified permanent truth.

The code environment is time-dependent.

Therefore:

Code Version + Jurisdiction + Effective Date

forms part of the project state.

---

15. CODE MANIFEST

Each project revision shall contain a Code Manifest.

The Code Manifest shall identify the regulatory inputs used by the generation engine.

Conceptually:

```
CODE_MANIFEST = HASH(JURISDICTION + REGULATORY_SOURCES + VERSIONS +
EFFECTIVE_DATES + RETRIEVAL_METADATA)
```

The resulting identifier becomes part of the project audit record.

Example:

```
CODE-MANIFEST-7F92...
```

The purpose is provenance.

It allows OneDraft to determine which regulatory dataset informed a particular project revision.

The manifest does not itself constitute certification of regulatory compliance.

---

16. CODE RULE OBJECTS

Individual requirements extracted from regulatory sources shall become structured Code Rule Objects.

Each rule shall contain:

Rule_ID

Source

Citation

Jurisdiction

Effective_Date

Rule_Type

Requirement

Applicability Conditions

Threshold

Unit

Affected Elements

Verification Method

Verification Status

Exception / Alternative Path

This permits the system to reason about applicability rather than applying every rule indiscriminately.

---

17. DESIGN VALIDATION ENGINE

The Design Validation Engine shall continuously evaluate the project model.

Validation categories shall include:

Geometric Validation

Spatial Validation

Dimensional Validation

Accessibility Validation

Egress Validation

Life-Safety Validation

Code Validation

Site Validation

Drawing Coordination

Schedule Coordination

Cross-Reference Validation

Customer Requirement Validation

Internal Consistency

Each validation produces a status:

PASS

FAIL

WARNING

REVIEW REQUIRED

NOT APPLICABLE

The system shall preserve the validation history.

---

18. DRAWING MODEL

Drawings shall be generated views of the project model.

Each drawing receives:

`Drawing_ID`

Each drawing contains:

Sheet Number

Sheet Title

Drawing Type

Scale

Orientation

View References

Model References

Dimensions

Annotations

Tags

Schedules

Details

Notes

Revision Information

Code References

Drawing Status

The drawing is therefore a controlled output rather than the primary project database.

---

19. SHEET SET MODEL

A project shall contain a Drawing Set.

The Drawing Set shall maintain:

Sheet Index

Sheet Numbering

Sheet Titles

Discipline

Drawing Status

Revision

Issue Date

Model Version

Code Manifest

Prepared By

Reviewed By

Approval Status

For architectural projects this may produce:

`A000`

`A001`

`A100`

`A110`

`A120`

`A200`

`A300`

`A400`

etc.

The system shall permit project-specific sheet numbering conventions.

---

20. VIEW MODEL

A single model can generate multiple views.

Views may include:

Floor Plan

Roof Plan

Foundation Plan

Reflected Ceiling Plan

Elevation

Section

Detail

3D Axonometric

Perspective

Site Plan

Enlarged Plan

Diagram

Each view references the underlying model.

Therefore:

One Wall

may simultaneously appear in:

Floor Plan
Section
Elevation
Detail
3D View

without becoming five independent walls.

---

21. DIMENSION MODEL

Dimensions shall be associated with model geometry.

A dimension shall contain:

Dimension_ID

Referenced Geometry

Dimension Type

Value

Unit

Tolerance

Display Style

Drawing References

Verification Status

Dimensions should be generated from geometry wherever possible rather than manually typed.

This prevents discrepancies between the model and documentation.

---

22. ANNOTATION MODEL

Annotations shall include:

Room Tags

Door Tags

Window Tags

Wall Tags

Detail References

Section References

Elevation References

Grid References

Level Markers

Keynotes

General Notes

Annotations must retain their relationship to the objects they describe.

---

23. REVISION MODEL

Every modification produces a new model revision.

Example:

`REV-000`

Initial project

`REV-001`

Customer modification

`REV-002`

Code adjustment

`REV-003`

Redline revision

`REV-004`

Final coordinated draft

A revision shall contain:

Revision_ID

Parent Revision

Author

Timestamp

Changed Elements

Changed Requirements

Changed Drawings

Changed Code Manifest

Validation Results

Redline Status

Customer Approval Status

The historical state shall remain recoverable.

---

24. REDLINE MODEL

Redlines shall not simply be graphical markup.

Each redline receives:

```
Redline_ID
```

and contains:

Source Drawing

Affected Element

Requested Change

Customer Instruction

Priority

Status

Created By

Created Date

Resolution

Resolved By

Resolved Date

A redline can therefore progress through:

OPEN

→ **UNDER REVIEW**

→ **IMPLEMENTED**

→ **VALIDATED**

→ **ACCEPTED**

or:

OPEN

→ **REJECTED / REQUIRES CLARIFICATION**

This creates a controlled revision process.

---

25. CONFLICT ENGINE

OneDraft shall detect conflicts generated by changes.

For example, moving a wall may affect:

Room Area

Door Position

Window Position

Electrical Fixtures

Mechanical Ductwork

Plumbing

Structural Elements

Dimensions

Sections

Elevations

Schedules

Accessibility

Egress

The system shall identify affected dependencies before producing the revised document set.

---

26. DOCUMENT PROVENANCE

Every generated document shall contain a provenance relationship to:

Project_ID

Model_Version

Drawing_Version

Code_Manifest

Generation_Event

Revision

Validation_State

The system therefore knows:

> Which project generated this drawing?
>
> Which model generated it?
>
> Which code baseline informed it?
>
> Which revision produced it?
>
> Which validation state existed at generation?

This becomes the basis of the OneDraft audit trail.

---

27. CUSTOMER ACCEPTANCE

Final project acceptance shall be represented as a structured event.

The acceptance record shall identify:

Project

Revision

Drawing Set

Model Version

Code Manifest

Redline Resolution State

Outstanding Warnings

Customer Review Status

Acceptance Date

Customer Identity

Acceptance Record

The system should explicitly distinguish:

Draft Generated

from:

Customer Accepted

from:

Professionally Reviewed

from:

Municipally Approved

These are four different states.

---

28. EXPORT MODEL

The Project Model shall be capable of generating multiple deliverables.

Primary architectural outputs shall include:

PDF

CAD-compatible vector output

Image renderings

Schedules

Specifications

Drawing Index

Revision Register

Code Manifest

Project Report

Where appropriate, future implementations may support additional interoperable BIM/CAD formats.

---

29. PROJECT STATE MACHINE

Every OneDraft project shall have a controlled lifecycle:

INTAKE

→ **SPECIFICATION**

→ **SITE ANALYSIS**

→ **CODE ANALYSIS**

→ **PROGRAMMING**

→ **SCHEMATIC DESIGN**

→ **DESIGN DEVELOPMENT**

→ **MODEL COORDINATION**

→ **DOCUMENT PRODUCTION**

→ **CUSTOMER REDLINE**

→ **REVISION**

→ **FINAL DRAFT**

→ **CUSTOMER ACCEPTANCE**

→ **EXPORT**

→ **ARCHIVE**

The project may return to an earlier state without destroying previous states.

---

30. THE SINGLE SOURCE OF TRUTH

The fundamental OneDraft architectural principle is:

THE PROJECT MODEL IS THE SOURCE OF TRUTH.

The drawings are derived representations.

The schedules are derived representations.

The dimensions are derived representations.

The sections are derived representations.

The elevations are derived representations.

The renderings are derived representations.

The customer-facing document package is derived from the same underlying project state.

This principle is what permits CAD-AI to maintain coordination across an entire project.

---

31. ARIA'S ROLE

Aria Powered shall function as the reasoning and orchestration layer operating upon the OneDraft Project Model.

Aria shall be capable of:

Interpreting specifications

Resolving structured requirements

Constructing design relationships

Proposing geometry

Applying constraints

Interpreting applicable regulatory controls

Generating documentation instructions

Detecting conflicts

Responding to redlines

Regenerating affected outputs

Explaining design decisions

Maintaining project continuity

Aria shall not be treated as the permanent database.

The Project Model is persistent.

Aria is the intelligent computational interface operating upon that model.

This distinction is essential for scalability.

---

32. CORE ARCHITECTURE

The resulting OneDraft architecture is therefore:

CUSTOMER

↓

ONEDRAFT INTERFACE

↓

PROJECT SPECIFICATION ENGINE

↓

REQUIREMENTS MODEL

↓

SITE + PROGRAM + CODE MODELS

↓

ONEDRAFT PROJECT MODEL

↓

ARIA REASONING / ORCHESTRATION ENGINE

↓

PARAMETRIC GEOMETRY ENGINE

↓

VALIDATION / CONFLICT ENGINE

↓

DRAWING GENERATION ENGINE

↓

DOCUMENT SET

↓

CUSTOMER REDLINE

↓

REVISION ENGINE

↓

VALIDATION

↓

FINAL DRAFT

↓

CUSTOMER ACCEPTANCE

↓

33. FOUNDATIONAL PRINCIPLE

OneDraft shall not be designed as an AI that merely "draws buildings."

It shall be designed as an **intelligent architectural documentation system that understands the relationships within a building project and generates coordinated documentation from those relationships.**

That distinction defines the technical foundation of CAD-AI — Aria Powered.

Version 0.1 — Foundational Project Data Model established.